Table of Contents

1.INTRODUCTION 2

1.1. Purpose of the Project 2

1.2. Scope 2

1.3. Applicable Standards and Documents 2

2.HAVEQUICK II INTERFACE OVERVIEW 3

2.1. System Architecture and Block Diagram 3

2.2.Manchester II Biphase Encoder 5

2.2.1. Principle of Manchester Encoding 5

2.2.2. Generation of Encoded Signal 5

2.3.Have Quick II Time-of-Day (TOD) Message Structure 6

2.4.Modified Hamming Code (4:8) 8

2.5.Binary To BCD Converter 10

3.GENERATIONOF TIME-OF-DAY MESSAGE 12

3.1. Overview 12

3.2. UTC Time Generation from GPS Week and TOW 13

3.3.TFOM(Time Figure Of Merit) 15

4.FPGA IMPLEMENTATION DETAILS 16

4.1. Manchester Encoder Module and Internal FSM 16

4.1. Hamming Encoder and Product Assembly 18

REFERENCES 20

1.INTRODUCTION

1.1. Purpose of the Project

The purpose of this project is to implement a **Have Quick II Time-of-Day (TOD) Interface** on an FPGA-based **GNSS L1/CA receiver**, compliant with the GPS ICD-060A standards. The primary goal is to generate precise, reliable TOD messages synchronized with GNSS signals. These TOD messages allow precise time dissemination and synchronization for systems utilizing GNSS-based timing information. The design includes both Hamming encoding, for error detection and correction, and Manchester encoding, to ensure robust and accurate data transmission.

1.2. Scope

This document provides comprehensive details on the FPGA design and implementation processes of the Have Quick II TOD interface tailored specifically for integration with GNSS L1/CA receiver systems. It covers TOD message structure, encoding methods, FPGA resource utilization, synchronization of the data generation to the GNSS receiver's 1PPS timing pulse, and thorough verification processes including functional simulation and hardware-level validation. The document also discusses technical challenges, design decisions, and ensures the implemented interface meets the accuracy and reliability criteria defined by the relevant standards.

1.3. Applicable Standards and Documents

The FPGA design and development processes described in this document comply with the following applicable standards and references:

GPS ICD-060A (Interface Control Document)
Specifies GNSS timing message formats, particularly the exact structure of TOD messages, and timing synchronization requirements for GNSS-GPS receivers.

Manchester Encoding Standard (MIL-STD-1553)
Describes Manchester encoding signal structures, timing specifications, and electrical requirements ensuring clear and unambiguous data transfer.

2.HAVEQUICK II INTERFACE OVERVIEW

The HAVEQUICK II interface is designed to transmit precise Time-of-Day (TOD) information in a structured and robust format, suitable for synchronization of GNSS receiver systems. This section provides a comprehensive overview of the TOD message structure, including its internal composition and encoding mechanisms as defined in ICD-GPS-060A.

The TOD message consists of several time-related fields such as hours, minutes, seconds, day of year, year, and a time quality indicator known as TFOM. These fields are serialized and transmitted using two distinct encoding schemes: **Modified Hamming Code**, which provides single-bit error correction capability, and **Manchester II (biphase-level) encoding**, which ensures signal robustness and simplifies clock recovery at the receiver end.

The following subsections detail the layout and bit composition of the message, the application of the modified Hamming code to each time field, and the characteristics of Manchester II encoding used during message transmission. Together, these components form the foundation of a timing interface that ensures reliable, error-tolerant delivery of time data with sub-millisecond alignment to GNSS-derived reference pulses

2.1. System Architecture and Block Diagram

The HQII timing interface is built on a fully hardware-driven FPGA architecture that processes GNSS-derived time with sub-millisecond precision and delivers it in a synchronized format. The system receives the GPS Week Number and Time of Week, converts them to UTC, formats the resulting time fields into BCD, applies Hamming (8,4) encoding, and finally transmits the complete message using Manchester encoding.

All operations are initiated precisely at the rising edge of the 1PPS signal. While the preamble is being transmitted, the system concurrently prepares the encoded time message, ensuring perfect alignment with the transmission protocol. The following block diagram illustrates the interconnection of the modules and the flow of time data throughout the system.

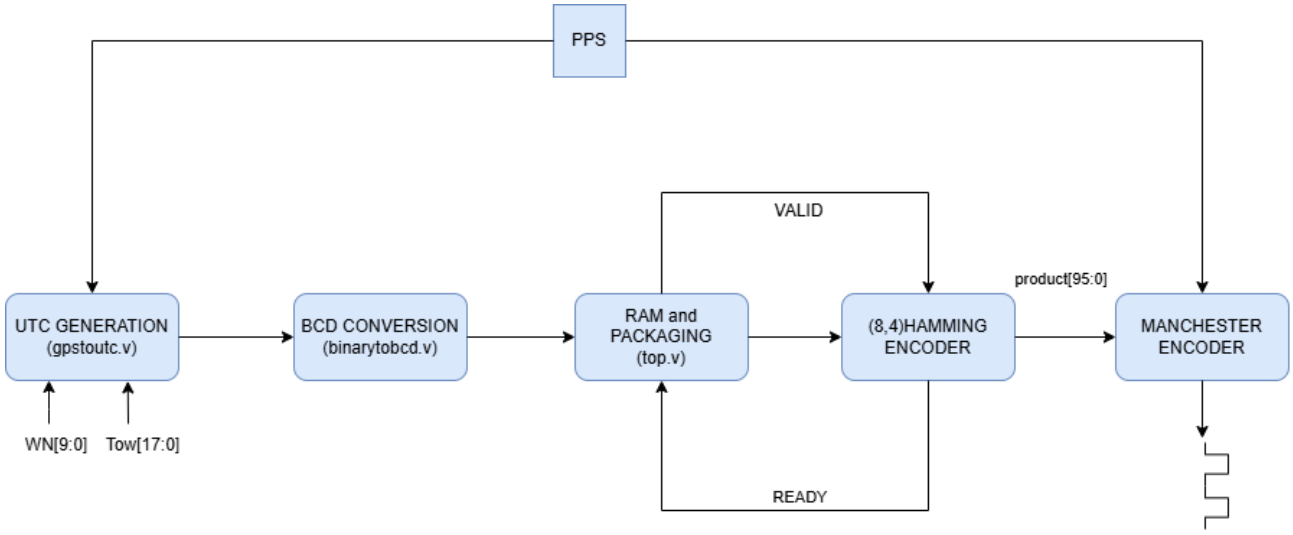

Figure 2.1 : Top-Level Block Diagram of the HQII Timing Interface

The diagram illustrates the core modules involved in generating and transmitting a Manchester-encoded HAVEQUICK II Time-of-Day (TOD) message. The process is triggered by the rising edge of the 1PPS signal generated by the internal GNSS receiver, which serves as the primary timing reference.

Upon this trigger, two processes are launched in perfect parallelism: the Manchester Encoder begins transmitting a 400-bit preamble of continuous logical ones, while the rest of the system concurrently prepares the final TOD message. This design eliminates sequential bottlenecks by allowing the time-critical Manchester output to proceed immediately, even as the actual data is still being encoded and assembled.

While the preamble is being sent, GPS Week Number and Time of Week inputs are converted to UTC, then formatted into BCD and written to RAM. Once all time fields are stored, a valid signal enables the Hamming Encoder, which encodes each 4-bit segment into 8-bit Hamming-coded output. These encoded bytes are accumulated in a 96-bit product register.

By the time the preamble finishes transmission, the product is fully formed and ready for output. The Manchester Encoder then seamlessly transitions to serializing the message bits, each following the standard HQII format: a '1' is represented as 300 µs high followed by 300 µs low, and a '0' as 300 µs low followed by 300 µs high.

This parallel structure not only improves efficiency but also ensures strict timing alignment with the 1PPS reference—meeting the HQII requirement of initiating transmission within ±10 µs of the PPS edge.

2.2.Manchester II Biphase Encoder

"The HAVE QUICK signal employs biphase (Manchester II) transmission at approximately 1667 bits per second. A one is defined as 300 microseconds of a high state followed by 300 microseconds of a low state. A zero is defined as 300 microseconds of a low state follow-d by 300 microseconds of a high state. A transmission of 240 miliseconds of logical ones(400 bits) precedes the start of message indicator(16 bits) and time, day of year, year and TFOM message (96 bits)." (ICD-GPS-060A). Manchester encoding ensures a transition in every bit period, making it self-clocking. This improves timing recovery and reduces bit synchronization errors. Unlike NRZ, it provides robust transmission even without a separate clock signal. Implementation details will be presented in the following sections of this report.

2.2.1. Principle of Manchester Encoding

Manchester encoding, also referred to as biphase-level modulation, is a line coding technique in which each data bit is represented by a transition in signal level occurring at the midpoint of the bit period. According to the convention used in ICD-GPS-060A, a logical '1' is represented by a high-to-low transition, and a logical '0' is represented by a low-to-high transition.

This ensures that every bit contains at least one level transition, providing two critical benefits:

- Self-clocking capability, allowing the receiver to recover timing information directly from the signal,
- Transition density, ensuring signal integrity and simplifying synchronization.

2.2.2. Generation of Encoded Signal

In this design, the Manchester encoder is implemented as a combinational-output, synchronous logic block that operates with a system clock. The encoder receives a sequence of binary input bits and converts each bit into a two-phase waveform:

- For a '1' bit: first half high**,** second half low,

- For a '0' bit: first half low, second half high.

The modulation is performed in real-time using an internal state machine. Timing is maintained by a precise counter that defines a **600 µs** bit period, with each half-cycle lasting **300 µs**.

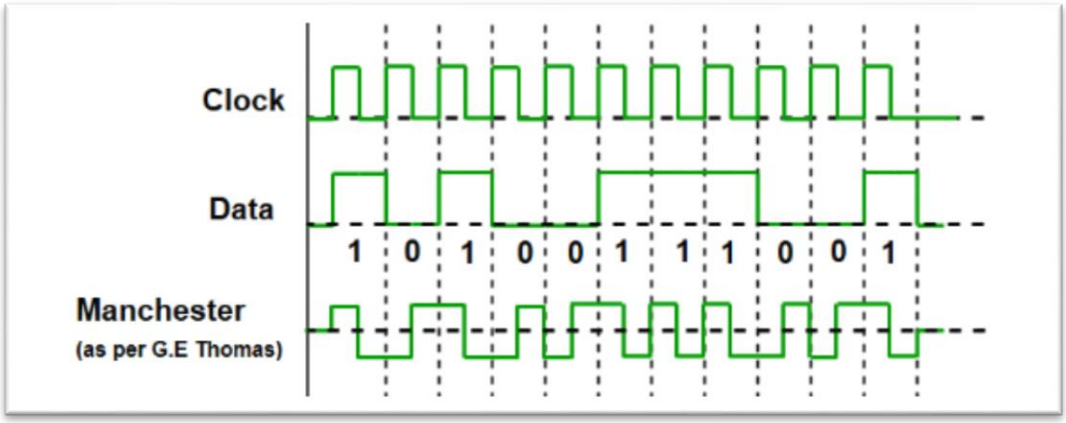

Figure 2.2 : Manchester Encoding Timing Diagram

2.3.Have Quick II Time-of-Day (TOD) Message Structure

Have Quick II standard defines a specific Time-of-Day (TOD) message structure that provides accurate and synchronized time information. The TOD message consists of data fields including hours, minutes, seconds, day of the year (DOY), year, and TFOM (Time Figure of Merit). Each data field is encoded using Hamming encoding for error detection and correction, and subsequently Manchester encoded for robust data transmission. The message generation begins precisely leading edge of a GNSS-derived 1PPS signal, ensuring consistent synchronization with GNSS time references.

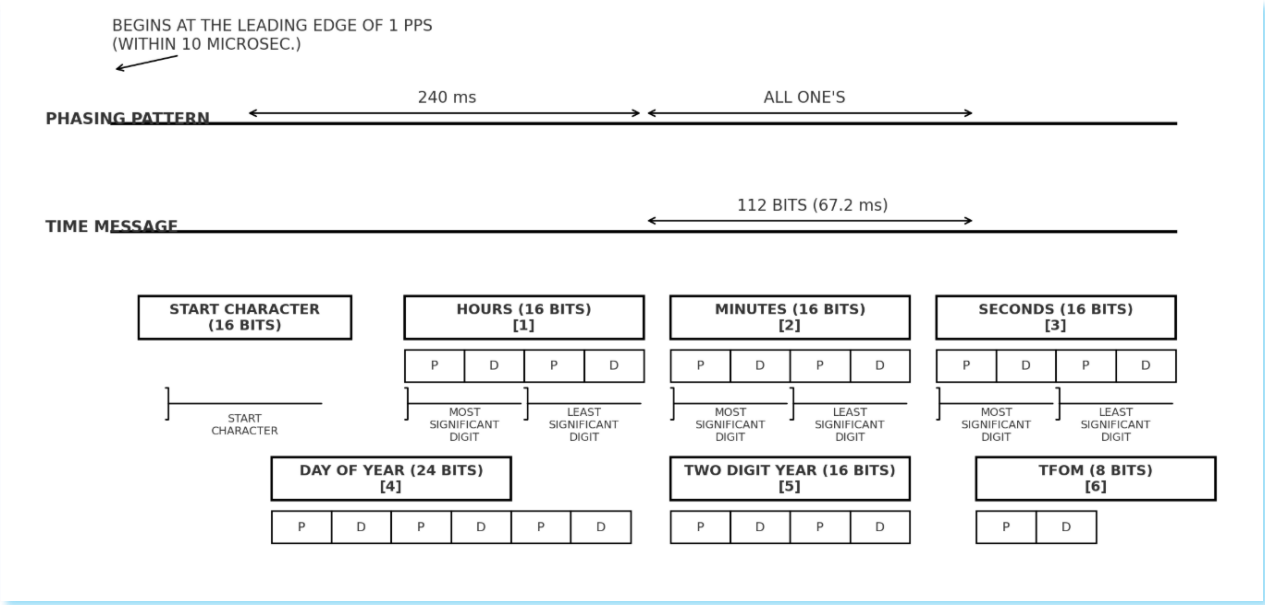

Figure 2.3 : HAVE QUICK TOD OUTPUT FORMAT AND TIMING

The figure above illustrates the transmission layout of the Have Quick II Time-of-Day (TOD) message, which is aligned with the rising edge of the GNSS-derived 1PPS signal. It consists of two main parts: the **Phasing Pattern** and the **Time Message**

At the beginning of the message, a Phasing Pattern (or preamble) of 400 bits is transmitted. This section enables the receiver to detect and synchronize with the incoming data stream. The interfacing equipment detects the preamble of 400 “logical ones” and aligns itself to the bit boundaries through a phase-lock loop.

Since the transmission uses Manchester II (Biphase-Level) encoding at a data rate of 1667 bits per second, the total duration of the **Phasing Pattern** is calculated :

$$T(preamble) = 400\ bits\ \div\ 1667\frac{bits}{sec} = 0.2399\text{sec} \approx 240\ ms$$

The second part is the actual **Time Message**, which consists of:

- Start Character (16 bits)
- Hours (16 bits)

- Minutes (16 bits)
- Seconds (16 bits)
- Day of Year – DOY (24 bits)
- Two Digit Year (16 bits)
- TFOM (8 bits)

The total number of bits in this section is:

$$N(message) = 16 + 16 + 16 + 24 + 16 + 8 + 16 = 112\ bits$$

Using the same Manchester encoding rate, the time required to transmit this section is:

$$t(message) = 112\ bits \div 1667\frac{bits}{\text{sec}} = 0.0672\text{sec} = 67.2\ ms$$

The overall transmission time of the entire Have Quick II TOD message is therefore:

$$t_{\text{o}}_{\text{a}} = 240\ ms + 67.2\ ms = 307.2\ ms$$

This transmission is initiated exactly at the leading edge of the 1PPS pulse, with a precision of ±10 µs, ensuring tight synchronization with UTC time. The use of Manchester II encoding improves timing robustness and simplifies receiver clock recovery due to guaranteed transitions in every bit period.

2.4.Modified Hamming Code (4:8)

"The HAVE QUICX data message uses a modified Hamning Code employing 4 parity bits and 4 data bits for each 8 bit character." (ICD-GPS-060A)

This coding scheme provides single-bit error correction capability, which ensures data integrity in environments where bit-level corruption is possible.

The summarized form of the encoding format as defined in the ICD is provided below:

Data Word	Code Word
d3 d0	b7 b0
0000	0 0 0 1 0 1 0 1
0001	0 0 0 0 0 0 1 0
0010	0 1 0 0 1 0 0 1
0011	0 1 0 1 1 1 1 0
0100	0 1 1 0 0 1 0 0
0101	0 1 1 1 0 0 1 1
0110	0 0 1 1 1 0 0 0
0111	0 0 1 0 1 1 1 1
1000	1 1 0 1 0 0 0 0
1001	1 1 0 0 0 1 1 1
1010	1 0 0 0 1 1 0 0
1011	1 0 0 1 1 0 1 1
1100	1 0 1 0 0 0 0 1
1101	1 0 1 1 0 1 1 0
1110	1 1 1 1 1 1 0 1
1111	1 1 1 0 1 0 1 0

Figure 2.4 : (8,4) Hamming Encoding For Each Number

The encoding operation shown in the table above can also be implemented in RTL using a case statement structure, where each 4-bit data word maps directly to its corresponding 8-bit code word. However, in our design, as shown in the code block below, we implemented the Hamming (8,4) encoder using a purely combinational logic approach, where parity bits are calculated dynamically based on XOR operations over the input bits. This method offers improved flexibility and hardware efficiency.

```
    input  wire        [3:0]          data,
    input  wire                       done,
    output reg                        ready = 1,
    /*-------------------- Output --------------------------------*/
    output wire        [7:0]          encoded,
    output reg [MAX_BYTES*8-1:0]      product = 0,
    output reg [3:0] byte_cnt = 0,
    output reg                        product_valid = 0
);
    /*-------------------- Hamming (8,4) encoder  ---------------------*/
```

```
wire p1 = data[0] ^ data[1] ^ data[3];
wire p2 = data[0] ^ data[2] ^ data[3];
wire p4 = data[1] ^ data[2] ^ data[3];
wire p8 = p1 ^ p2 ^ p4 ^ data[0] ^ data[1] ^ data[2] ^ data[3];
wire [7:0] enc_next = {p8, data[3], data[2], data[1], p4, data[0], p2, p1};
assign encoded = enc_next;
```

Code Snippet 2.1: RTL Design For (8,4) Hamming Code.

As shown in the code snippet, the block is implemented entirely as a combinational circuit. It communicates with the upstream module through a **valid–ready** handshake mechanism. This design choice enables the encoder to process multiple data words within a single clock cycle, increasing throughput and minimizing latency.. The Start of Message Indicator is transmitted as a fixed 16-bit character and does not utilize Hamming (8,4) encoding. Unlike the time fields, this segment is sent in raw binary form and is excluded from the error correction process.

Each time-related field (hours, minutes, seconds, day of year, year) is first generated in binary format and then converted to **Binary-Coded Decimal (BCD)** prior to encoding. This conversion ensures that each digit of the decimal value is represented by a separate 4-bit nibble, in accordance with the ICD-GPS-060A structure. The use of BCD formatting allows for consistent encoding using Modified Hamming (8,4) code, as each 8-bit character ultimately represents a single decimal digit. For this reason, dedicated **Binary-to-BCD** conversion units have been implemented within the design.

2.5.Binary To BCD Converter

The Binary_to_BCD module converts a binary number to its **Binary-Coded Decimal (BCD)** representation using the **Double Dabble** algorithm (also known as shift-and-add-3). The module is fully synchronous and operates on the rising edge of i_Clock.The module uses a **finite state machine (FSM)** with six states to carry out the conversion:

1.s_IDLE:

Waits for i_Start to go high. On trigger, it initializes internal registers and transitions to the shift phase.**2.s_SHIFT:**

Shifts the most significant bit (MSB) of the binary input into the least significant bit (LSB) of the BCD vector.

3.s_CHECK_SHIFT_INDEX:

Determines whether all bits from the binary input have been processed. If not, it proceeds to digit correction.

4.s_ADD:

For each 4-bit BCD digit, checks if the value exceeds 4. If so, 3 is added to correct the digit per the Double Dabble algorithm.

5.s_CHECK_DIGIT_INDEX:

Iterates through all BCD digits. If completed, returns to s_SHIFT for the next binary bit.

6.s_BCD_DONE:

Sets the output valid flag (o_DV) and holds the result in o_BCD.

The input width of this module is designed to be fully parameterizable. For instance, when converting the Day of Year (DOY) field—which is 12 bits wide—the INPUT_WIDTH parameter is set to 12 accordingly.

In the current design, a separate converter instance is used for each time-related value (e.g., hours, minutes, seconds, DOY, year). While this approach increases logic resource usage, it significantly improves performance by allowing all conversions to occur in parallel. Assuming each conversion takes approximately 6 clock cycles to complete, sequentially converting each value would introduce considerable latency.

Therefore, parallelization has been prioritized for speed in this version of the design, though this architecture can be adjusted depending on system requirements and resource constraints.

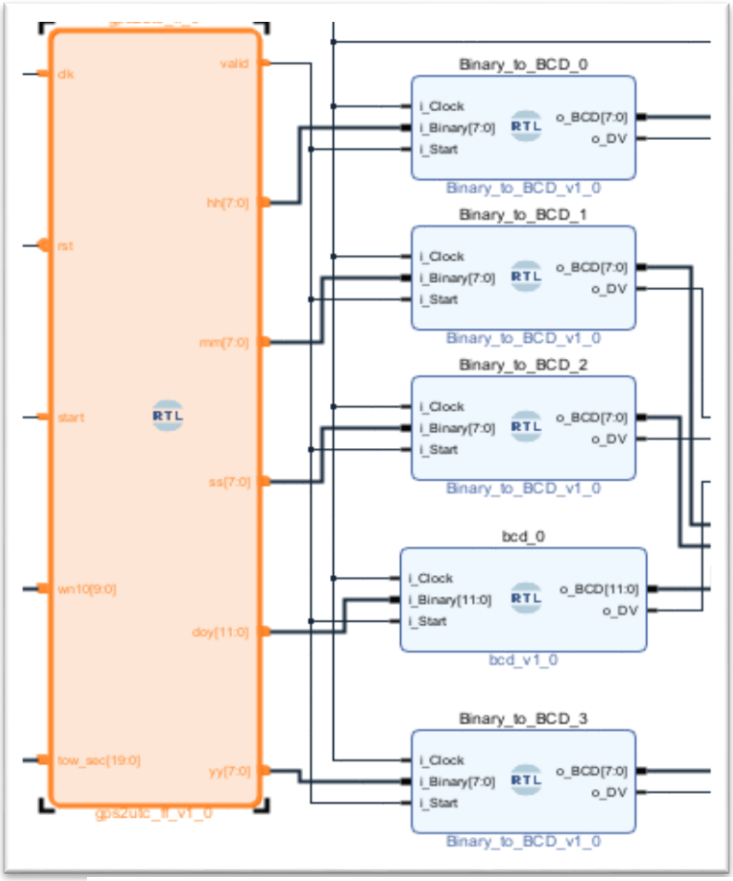

Figure2.3 :BCD Converters In RTL Block Design

3.GENERATIONOF TIME-OF-DAY MESSAGE

3.1. Overview

This section outlines the generation of the 112-bit TOD message starting from GPS time (Week Number and TOW). The time is converted to UTC, then split into fields (HH, MM, SS, DOY, YY), which are encoded in BCD and Hamming (8,4) formats. A fixed start character and a TFOM field—indicating time accuracy—are appended. The final message is serialized and Manchester encoded, aligned with the 1PPS signal.

3.2. UTC Time Generation from GPS Week and TOW

The UTC generation process constitutes the first and most critical stage in the Time-of-Day (TOD) message pipeline. Since the GPS system operates on its own continuous time scale—which does not account for leap seconds and uses a rolling 10-bit Week Number—the conversion to UTC is essential to produce a synchronized and universally accepted time reference.

In this implementation, the module `gps2utc_ff` receives two primary inputs from the GNSS core: the 10-bit **GPS Week Number** (`wn10`) and the 17-bit **Time of Week in seconds** (`tow_sec`). To handle **week number rollover**, which occurs every 1024 weeks (~19.7 years), a 2-bit **epoch index** parameter (`EPOCH_INDEX`) is prepended, forming a 12-bit extended week number. This allows the system to operate continuously across multiple GPS eras, e.g., 1980–1999, 1999–2019, 2019–2038.

The total number of seconds since the GPS epoch is calculated using:

$$gps_sec = (EPOCH_INDEX \cdot 1024 + wn10) \times 604800 + tow_sec \text{ x } 6$$

A fixed **leap second offset** (`LEAP_OFFSET`) is then subtracted to produce UTC seconds:

$$utc_sec = gps_sec - \Delta t_\{LS\}$$

This UTC second value is then broken down into:

Time-of-day fields: hours, minutes, and seconds, derived from modulo operations on the number of seconds per day.

Calendar date: derived using the **Fliegel–Van Flandern algorithm**, a closed-form calculation which converts Julian Day Numbers (JD) into Gregorian calendar components.

The resulting output fields are:

- `hh` – UTC hour (0–23)

- `mm` – UTC minute (0–59)
- `ss` – UTC second (0–59)
- `doy` – Day of year (1–366)
- `yy` – Two-digit UTC year (00–99)

All calculations are done in **combinational logic**, and the final values are latched on the rising edge of the system clock upon assertion of the `start` pulse, which is synchronized to the incoming 1PPS signal. A one-clock-cycle `valid` pulse marks the completion of the conversion.This module is fully autonomous, uses no loops or multipliers, and is optimized for real-time FPGA implementation. It ensures the temporal alignment of the outgoing TOD message with the GNSS-based UTC reference, forming the foundation of timing precision in the HQII interface.The module is designed to operate autonomously and can be easily instantiated in any design by providing two standard GNSS inputs: tow_sec (Time of Week in seconds) and wn10 (10-bit GPS Week Number). No additional external logic is required. The UTC time outputs are directly generated on the rising edge of the system clock, synchronized to a one-cycle start pulse (typically aligned with 1PPS).

```
wire [31:0] gps_sec = wn_full * SEC_WEEK + tow_sec ;
wire [31:0] utc_sec = gps_sec - LEAP_OFFSET;
wire [31:0] sec_day  = utc_sec % SEC_DAY;
wire [5:0]  hh_c     = sec_day / 32'd3600;
wire [5:0]  mm_c     = (sec_day / 32'd60) % 6'd60;
wire [5:0]  ss_c     = sec_day % 32'd60;
wire [31:0] days     = utc_sec / SEC_DAY; // Fliegel-Van Flandern (kapalı form)
wire [31:0] JDp      = days + JD0;
wire [31:0] L1       = JDp + A;
wire [31:0] N1       = (4*L1) / B;
wire [31:0] L2       = L1 - ((B*N1 + 3)/4);
wire [31:0] I1       = (4000*(L2+1)) / C;
wire [31:0] L3       = L2 - ((D*I1)/4) + 31;
wire [31:0] J1       = (F*L3) / E;
```

```
wire [31:0] Day_c    = L3 - ((E*J1)/F) + 1;
wire [31:0] K1       = J1 / 11;
wire [31:0] Month_c  = J1 + 2 - 12*K1;
wire [15:0] Year_c   = 100*(N1-49) + I1 + K1;
```

Code Snippet3.1 : Fliegel-Van Flandern Algortihm For Generating UTC Time

3.3.TFOM(Time Figure Of Merit)

"The Time Figure of Merit (TFOM) denotes the quality of the time solution as a discrete integer from 0-9." (ICD-GPS-060A)

The Time Figure of Merit (TFOM) is an integer value ranging from 0 to 9 that indicates the quality and confidence level of the system's current time solution. Lower values (closer to 0) represent higher accuracy and reliability of the time information, while higher values indicate increasing uncertainty or degraded time validity.

TFOM is commonly used in military and timing-critical systems to signal whether the current time estimate can be trusted for synchronization or coordination purposes. It is typically derived from a combination of signal integrity, data validity, timing model stability, and oscillator drift analysis.

Note: The specific meaning of each TFOM level may vary depending on system implementation and performance assumptions.

Since our system currently does not implement TFOM estimation, the TFOM field is populated with all zeros **(TFOM = 0)** in the transmitted data.

TFOM	Estimated Accuracy (with respect to	Description
All bits high	No information	Invalid or missing data
9	Greater than 10 milliseconnds	Low accuracy
8	100 microuseconds – 1 millisecond	Moderate accuracy
7	10 – 100 microseconds	
6	10 – 100 nanoseconds – 1 microsecond	
5	10 – 100 nanoseconds	
1	Better than 1 nanosecond	High precision
0	Proper/nominal operation	Used when external equipment cannot con

. **Figure 3.1 :**TFOM (Time Figure of Merit) Values And Their Corresponding Estimated Timing Accuracy With Respect to UTC

4.FPGA IMPLEMENTATION DETAILS

This section presents a detailed explanation of the internal operation of key functional blocks used in the HAVEQUICK II interface design. While earlier sections focused on high-level flow and message structure, this part is dedicated to the architectural implementation and timing control mechanisms that ensure deterministic and standard-compliant behavior.

Modules such as the finite state machine (FSM), Manchester encoder, message formatter, and product register are explained with attention to the control logic and signal transitions. Any low-level hardware behaviors or timing constraints that are not evident in the functional overview will be clarified here to support both verification and integration efforts.

4.1. Manchester Encoder Module and Internal FSM

Here, described the internal logic structure of the ManchesterEncoder_HQII module with a focus on its finite state machine (FSM), bit timing, and data scheduling mechanisms.

The FSM consists of three main states: IDLE, Valid_State, and Message_State. Upon detecting a rising edge on the start signal, the FSM transitions from IDLE to Valid_State, where a fixed 400-bit preamble (logic '1') is transmitted. Each bit is modulated over a 600 µs interval using Manchester II encoding. The bit duration is implemented using a 32-bit counter (counter_m), which increments on every clock cycle. The transition from Valid_State to Message_State occurs when counter_400 reaches 400.

In Message_State, the encoded 112-bit TOD message is read from the time_msg_r shift register. The message includes a 16-bit Start Character followed by 96 bits of time data and TFOM. The FSM uses the same counter_m for Manchester bit timing and a separate bit_index counter to iterate through the message bits. Each bit is output in two halves: the first half representing the inverse of the bit, and the second half representing the bit itself, producing the required mid-bit transition.

The transmission completes once all 112 bits have been modulated. At this point, the FSM resets its counters and returns to the `IDLE` state, ready for the next cycle. The design ensures deterministic output timing and supports one complete frame transmission per second, synchronized to the external 1PPS trigger.

The FSM (Finite State Machine) structure implemented within the module is illustrated below.

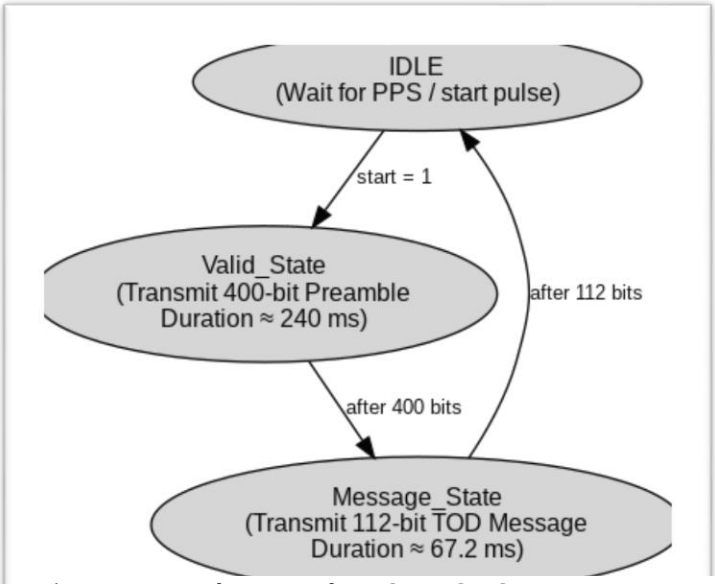

Figure 4.1 : ManchesterEncoder_HQII.v FSM Structre

4.2. Hamming Encoder and Product Assembly

The Hamming encoder module receives individual 4-bit BCD digits as input and encodes each into an 8-bit format using a modified Hamming (8,4) code. The resulting byte (enc_next) contains both data and parity bits, arranged in a predefined bit order. The encoding process is performed combinationally, without latency.Each encoded byte is written into the product register using a sequential handshake mechanism. When the ready signal is asserted and the done signal is low, the module accepts the current encoded output and writes it to the product register at a position determined by the current byte_cnt value. This results in a contiguous left-to-right accumulation of bytes within the register:

$$\begin{aligned} \text{product}[7{:}0] &\leftarrow \text{1st digit} \\ \text{product}[15{:}8] &\leftarrow \text{2nd digit} \\ &\ldots \\ \text{product}[95{:}88] &\leftarrow \text{12th digit} \end{aligned}$$

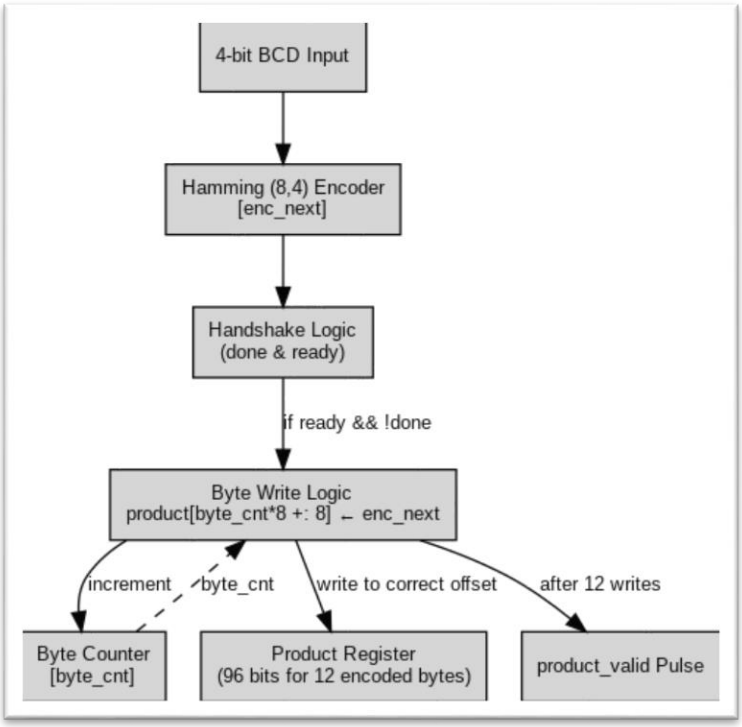

Figure 4.2 : Hamming Encoder and Product Register Write Flow.

The byte_cnt counter is incremented after each successful write. Once all digits are encoded (typically 12 BCD digits), the module asserts product_valid for a single clock cycle, indicating that the full 96-bit payload is ready. The system then resets byte_cnt to zero and waits for the next activation.

This mechanism guarantees that each encoded character occupies a fixed 8-bit slot in the output register and that all digits are written in the correct temporal order, ready for final framing and transmission.

REFERENCES

[1] NAVSTAR GPS Joint Program Office, *Interface Control Document – GPS User Equipment Precise Time and Time Interval (PTTI) Interface*, Drawing No. ICD-GPS-060, 2 June 1986.

[2]. *Jose, "Design of Manchester II bi-phase encoder for MIL-STD-1553 protocol,"* 2013 International Mutli-Conference on Automation, Computing, Communication, Control and Compressed Sensing (iMac4s), *Kottayam, India, 2013*,

[3] http://maia.usno.navy.mil/